

Sprint #0 Report

Instructions

Objectives

- Make decisions on the Solitaire software development project.
- Learn unit testing and GUI programming in the language of your choice.

Deliverables and Grading Policy

Read the “CS 449 Homework Overview” document **carefully** and make the key decisions for the software development. Use the following template to complete your report.

1. Key Decisions of the Solitaire Project (2 points)

Topic	Decision
Object-oriented programming language (if not using OOP, then justify the use of the language)	Gdscript
GUI library (strongly encouraged)	Built in
IDE (Integrated Development Environment)	Godot game engine
xUnit framework (e.g., JUnit for Java)	GUT (godot unit test)
Programming style guide (must read it carefully)	GDScript style guide
Project hosting site	Github.com
Other decisions if applicable	

Sample programming style guides:

- Google Java Style Guide: <https://google.github.io/styleguide/javaguide.html>
- Google C++ Style Guide: <https://google.github.io/styleguide/cppguide.html>
- Google Python Style Guide: <https://google.github.io/styleguide/pyguide.html>

2. Unit testing (4 points)

Find a tutorial on the unit test framework you have chosen and write at least two xUnit tests of a program you have written or found elsewhere. Attach here (1) the screenshot of your program execution and (2) the source code of your program.

For example, Junit5 user guide: <https://docs.junit.org/5.10.3/user-guide/index.html>

```
extends GutTest
```

```
func before_each():  
    gut.p("ran setup", 2)
```

```
func after_each():  
    gut.p("ran teardown", 2)
```

```
func before_all():
```

```
gut.p("ran run setup", 2)

func after_all():
    gut.p("ran run teardown", 2)

func test_assert_eq_number_not_equal():
    assert_eq(1, 2, "Should fail. 1 != 2")

func test_assert_eq_number_equal():
    assert_eq('asdf', 'asdf', "Should pass")

func test_assert_true_with_true():
    assert_true(true, "Should pass, true is true")

func test_assert_true_with_false():
    assert_true(false, "Should fail")

func test_something_else():
    assert_true(false, "didn't work")
```

```
res://test/unit/test_example.gd                                     2/5 passed
├─ test_assert_eq_number_not_equal                                fail
│   └─ fail: [1] expected to equal [2]: Should fail. 1 != 2    at line 15
├─ test_assert_true_with_false                                    fail
│   └─ fail: Should fail                                         at line 24
└─ test_something_else                                           fail
    └─ fail: didn't work                                          at line 27
```

```
res://test/unit/test_example.gd
fail * test_assert_eq_number_not_equal
      [Failed]: [1] expected to equal [2]: Should fail. 1 != 2
fail      at line 15
* test_assert_eq_number_equal
* test_assert_true_with_true
* test_assert_true_with_false
      [Failed]: Should fail
      at line 24
* test_something_else
      [Failed]: didn't work
      at line 27
2/5 passed.
```

```
=====
= Run Summary
=====
```

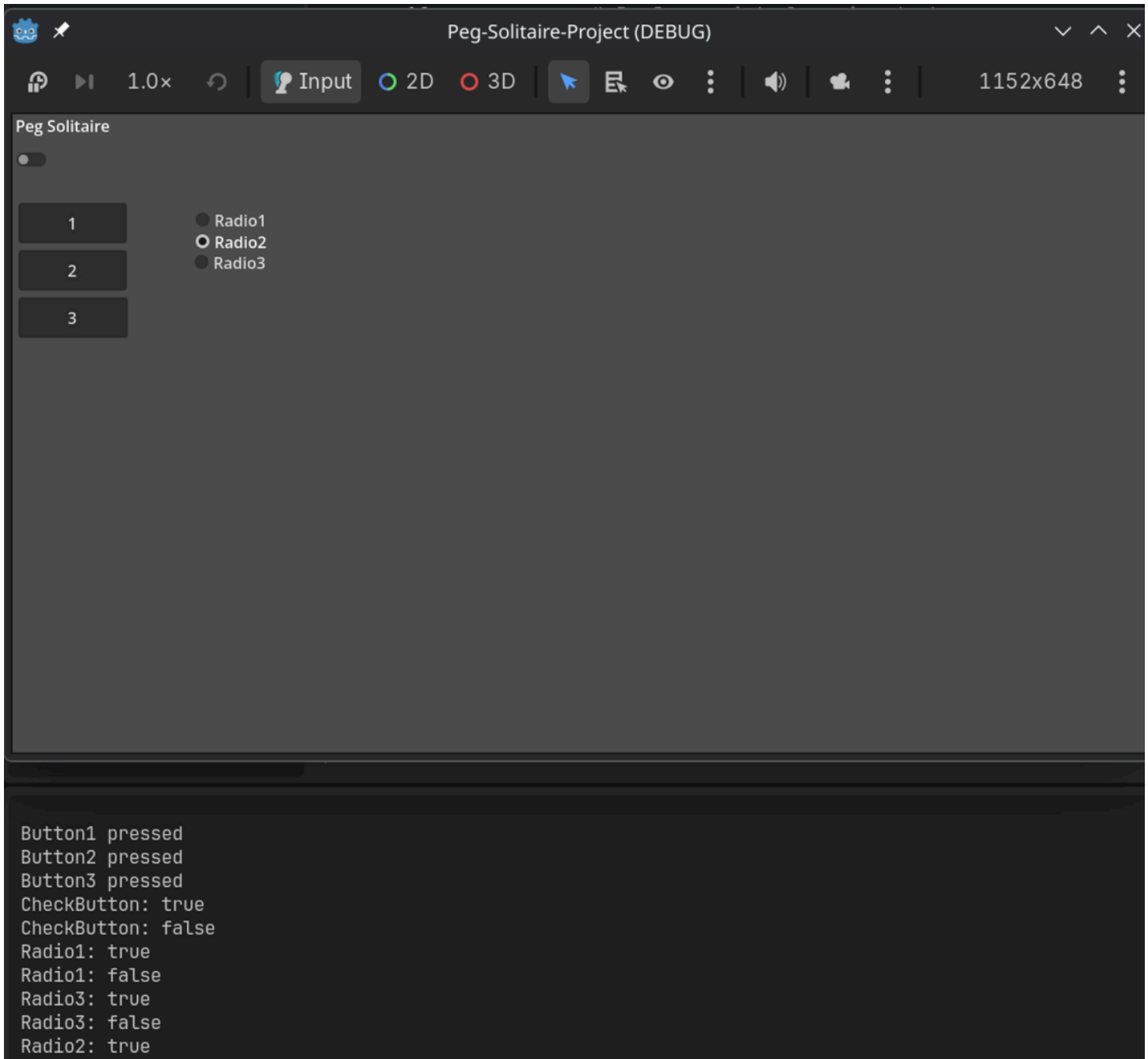
```
res://test/unit/test_example.gd
- test_assert_eq_number_not_equal
  [Failed]: [1] expected to equal [2]: Should fail. 1 != 2
    at line 15
- test_assert_true_with_false
  [Failed]: Should fail
    at line 24
- test_something_else
  [Failed]: didn't work
    at line 27
```

```
Totals
-----
Scripts                1
Tests                  5
Passing Tests          2
Failing Tests          3
Asserts                2/5
Time                   0.343s
```

3. GUI programming (4 points)

Write a GUI program in the language you have chosen for your Solitaire project. The GUI of your program must include text, lines, a check box, and radio buttons. In this assignment, any GUI program of your own work is acceptable.

Attach here (1) the screenshot of your program execution and (2) the source code of your program.



extends Control

Called when the node enters the scene tree for the first time.

func _ready() -> void:

var group:= ButtonGroup.new()

for i in \$Radio_Buttons.get_children():

if i is BaseButton:

i.button_group = group

```
i.toggle_mode = true
i.toggled.connect(_on_radio_toggled.bind(i.name))
```

```
# Called every frame. 'delta' is the elapsed time since the previous frame.
func _process(delta: float) -> void:
    pass
```

```
func _on_button1_pressed() -> void:
    print("Button1 pressed")
    pass # Replace with function body.
```

```
func _on_button2_pressed() -> void:
    print("Button2 pressed")
    pass # Replace with function body.
```

```
func _on_button3_pressed() -> void:
    print("Button3 pressed")
    pass # Replace with function body.
```

```
func _on_radio_toggled(toggled_on: bool, _extra_arg_0: String, _extra_arg_1: String) -> void:
    if toggled_on:
        print(_extra_arg_1 + ": true")
    else:
        print(_extra_arg_1 + ": false")
```

```
func _on_check_button_toggled(toggled_on: bool, extra_arg_0: String) -> void:
    if toggled_on:
        print(extra_arg_0 + ": true")
    else:
        print(extra_arg_0 + ": false")
    pass # Replace with function body.
```